Generación de Escenarios de Movilidad Urbana para la Toma de Decisiones en Localidades de Bogotá

Grupo 3

DOCUMENTO DE DISEÑO DE SOFTWARE (SDD)

20 de octubre de 2026

versión 1.0

Erick Salazar Suárez
Felipe Andrés Garrido Flores
Samuel Esteban Campos
Santiago Chitiva Contreras

Pontificia Universidad Javeriana · Facultad de Ingeniería · Ingeniería de Sistemas

1. Historial de Cambios

Versión	Fecha	Descripción	Responsables
1.0	20/10/2026	Primera versión del SDD. Describe la arquitectura del sistema tal como está implementada al cierre del Sprint 9: pipeline ETL, API, visor web, módulo de semáforos, simulador 3D comparativo y las mejoras de usabilidad de la versión 2.2 del SRS.	Todos

Tabla 1: Historial de cambios.

2. Tabla de Contenidos

1. Historial de Cambios 1
2. Tabla de Contenidos 2
3. Lista de Figuras 3
4. Lista de Tablas 3
5. Introducción 4
6. Arquitectura 4
   6.1. Vista Lógica 4
   6.2. Vista Física 6
   6.3. Vista de Procesos 7
7. Diseño Detallado 9
   7.1. Estructura 9
   7.2. Comportamiento 12
   7.3. Persistencia 13
   7.4. Interfaz de Usuario 14
8. Referencias 14

3. Lista de Figuras

Figura 1 Diagrama de componentes. 5
Figura 2 Diagrama de despliegue. 7
Figura 3 Actividades de una corrida del pipeline. 8
Figura 4 Estados de una corrida. 8
Figura 5 Bucle sincronizado (lockstep) de la comparación 3D. 9
Figura 6 Clases del pipeline y de las corridas. 10
Figura 7 Clases del módulo de semáforos. 10
Figura 8 Clases del simulador 3D comparativo. 11
Figura 9 Secuencia de una corrida parametrizada. 12
Figura 10 Secuencia de la comparación 3D. 12
Figura 11 Secuencia de la comparación de programas. 13
Figura 12 Modelo de persistencia en archivos. 13
Figura 13 Navegación del visor. 14

4. Lista de Tablas

Tabla 1 Historial de cambios. 1
Tabla 2 Endpoints de la API por grupo. 6
Tabla 3 Componentes del visor. 11
Tabla 4 Archivos del sistema y control de versiones. 13

5. Introducción

Este documento describe cómo está construido el sistema de generación de escenarios de movilidad para Usaquén. El SRS [1] dice *qué* debe hacer el sistema; el SDD explica *cómo* lo hace: en qué componentes se divide, dónde corre cada uno, cómo se comunican, qué clases tiene cada componente, qué guarda en disco y cómo navega el usuario por la interfaz. Sigue la estructura de la plantilla de la carrera, que a su vez toma las vistas 4+1 de Kruchten [2] y el estándar IEEE 1016 [3].

Alcance. Se cubre todo lo que está en el repositorio a octubre de 2026:

- los notebooks de exploración que producen la red y la evidencia en data/processed/;
- el paquete simulation-service (pipeline ETL sobre SUMO [4] y API en FastAPI [5]);
- el visor web kepler/ (React, kepler.gl [6] y three.js [7]).

El transporte público con GTFS y la validación GEH, que siguen pendientes según el SRS, se mencionan solo donde el diseño ya deja el lugar para ellos.

Convenciones. Los nombres de módulos, clases y funciones se escriben como en el código (GestorCorridas, run_pipeline). El código y sus comentarios están en español, igual que este documento. En los diagramas, las cajas con borde punteado son sistemas o servicios externos.

Relación con otros documentos. Los requerimientos citados (RF-xxx, RNF-xxx) son los del SRS versión 2.2. Los paquetes de trabajo (F1.1 a F5.6) son los del SPMP versión 2.2.

6. Arquitectura

El sistema es una aplicación local de tres capas con un motor de simulación externo:

1. **Datos y preparación:** notebooks de Jupyter que limpian la encuesta, construyen la red 3D y descargan la evidencia (CRISP-DM, Fases 1 y 2).
2. **Servicio de simulación:** un paquete de Python que genera la demanda, llama a SUMO y expone los resultados por HTTP y WebSocket.
3. **Visor web:** una aplicación de React que muestra los viajes en un mapa, deja parametrizar corridas, analizar semáforos y abrir el simulador 3D.

SUMO no es parte del código del proyecto: se instala con el paquete eclipse-sumo de pip y se usa como proceso externo (sumo, duarouter, netconvert) o controlado por TraCI [8].

6.1. Vista Lógica

La Figura 1 muestra los componentes y sus dependencias. Las flechas van del componente que usa al usado.

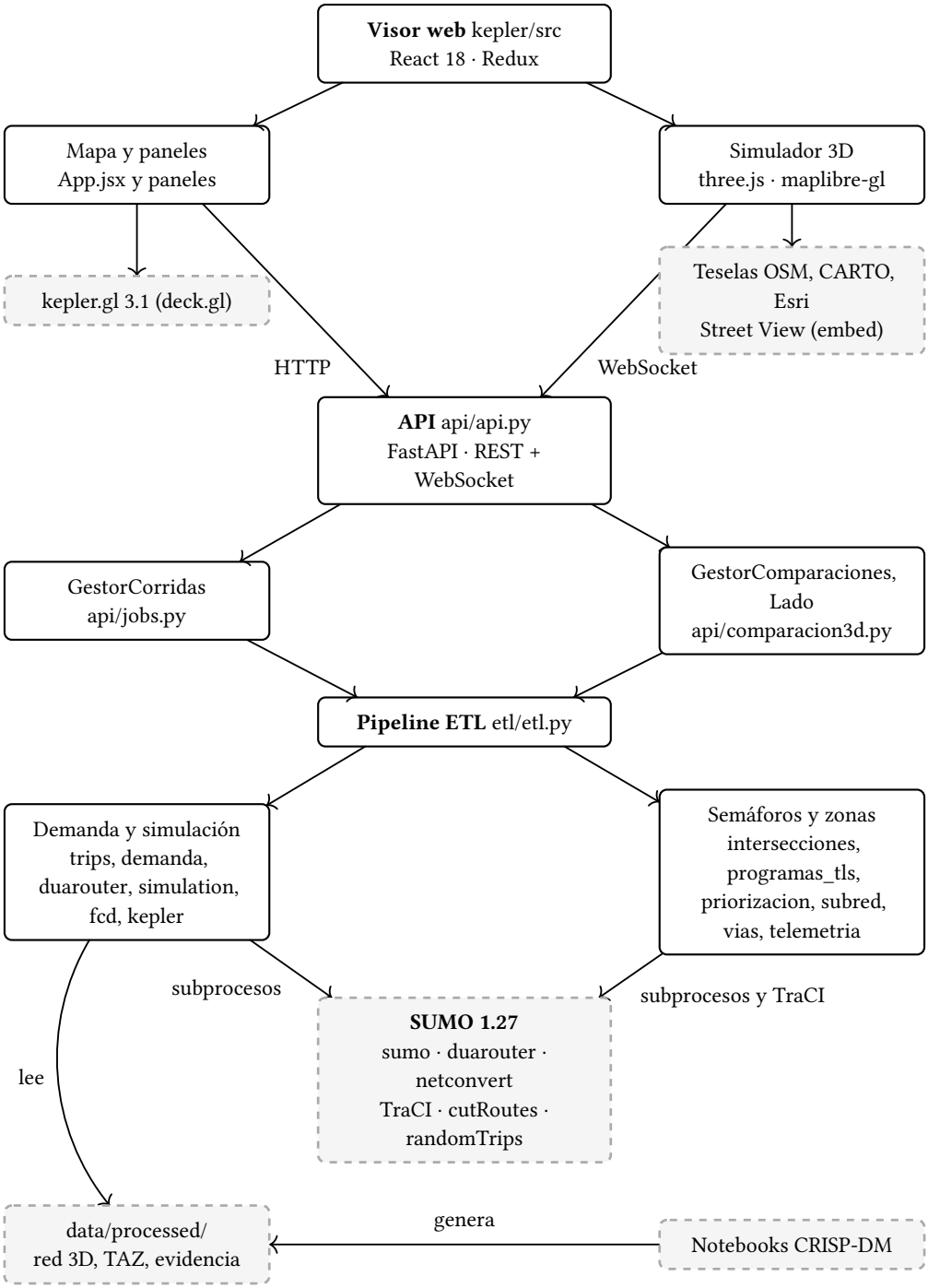


Figura 1: Diagrama de componentes.

Visor web. App.jsx (componente Visor) es dueño del mapa de kepler.gl y del estado global de la pantalla: la corrida cargada, la pestaña abierta, el cruce elegido, los cierres viales y el tema. Los paneles reciben ese estado por propiedades y avisan los cambios con funciones (onElegir, onChange, onEjecutar). api.js concentra todas las llamadas a la API; hooks.js lleva el historial de corridas y el sondeo cada 2 s. El simulador 3D se carga en diferido (React.lazy), porque three.js y maplibre pesan varios megabytes y la mayoría de las sesiones no lo abren.

API. Es una sola aplicación FastAPI. Los endpoints se agrupan por tema (Tabla 2). Las tareas largas no bloquean la petición: GestorCorridas corre una corrida a la vez en un ThreadPoolExecutor(1), GestorComparaciones prepara cada comparación 3D en un hilo y la comparación de programas corre en un hilo propio. Las pruebas reemplazan los gestores con app.dependency_overrides.

Grupo	Endpoints
Demanda y corridas	GET /demanda/modos, /demanda/presets, /demanda/estilos; POST /simulaciones, GET /simulaciones[/{id}], POST /simulaciones/{id}/cancelar, GET /simulaciones/{id}/kepler-trips
Precalculadas	GET /precalculadas, GET /precalculadas/{preset}/kepler-trips (gzip tal cual, con X-Tamano-Original)
Intersecciones	GET /intersecciones[/{id}], GET /intersecciones/priorizacion, POST /intersecciones/{id}/escenarios
Programas	GET /intersecciones/{id}/programas[/{prog}], PUT .../programas/{prog}, POST .../programas/webster, POST /intersecciones/{id}/comparaciones, GET /comparaciones/{id}
Red	GET /red/vias (vías para elegir cierres)
Simulador 3D	POST /cruces/{nodo}/comparacion3d, GET /comparacion3d/{id}[/geometria], DELETE /comparacion3d/{id}, WS /comparacion3d/{id}

Tabla 2: Endpoints de la API por grupo.

Pipeline ETL. etl/etl.py orquesta los pasos y los módulos de etl/sumo/ hacen cada uno. Todos siguen el mismo patrón: un *Config inmutable con los parámetros (y to_args() cuando llama a una herramienta de SUMO), un *Result con lo producido, un *Stats que se va llenando mientras corre y una excepción *Error(RuntimeError) propia. configuration.py es la única fuente de rutas y constantes (ScenarioPaths, HORA_INI, HORA_FIN, MAX_COPIAS).

Bibliotecas principales. En Python: FastAPI y Pydantic, traci y sumolib (fijados a la versión 1.27.1 de SUMO), pyproj para pasar las coordenadas de la red a longitud y latitud, pandas y geopandas en los notebooks. En el visor: React 18, Redux con el reductor de kepler.gl, three.js 0.169 para la escena 3D y maplibre-gl 4.7 para los minimapas y las vistas del cruce.

6.2. Vista Física

Todo corre en el computador del usuario (Figura 2). No hay base de datos ni servidor remoto: el estado vive en archivos. Los únicos servicios externos son de solo lectura y se usan desde el navegador: las teselas de los mapas base y el Street View embebido. La evidencia de priorización (siniestros, colegios, edificios) se descarga una vez con python -m etl.sumo.evidencia y queda versionada.

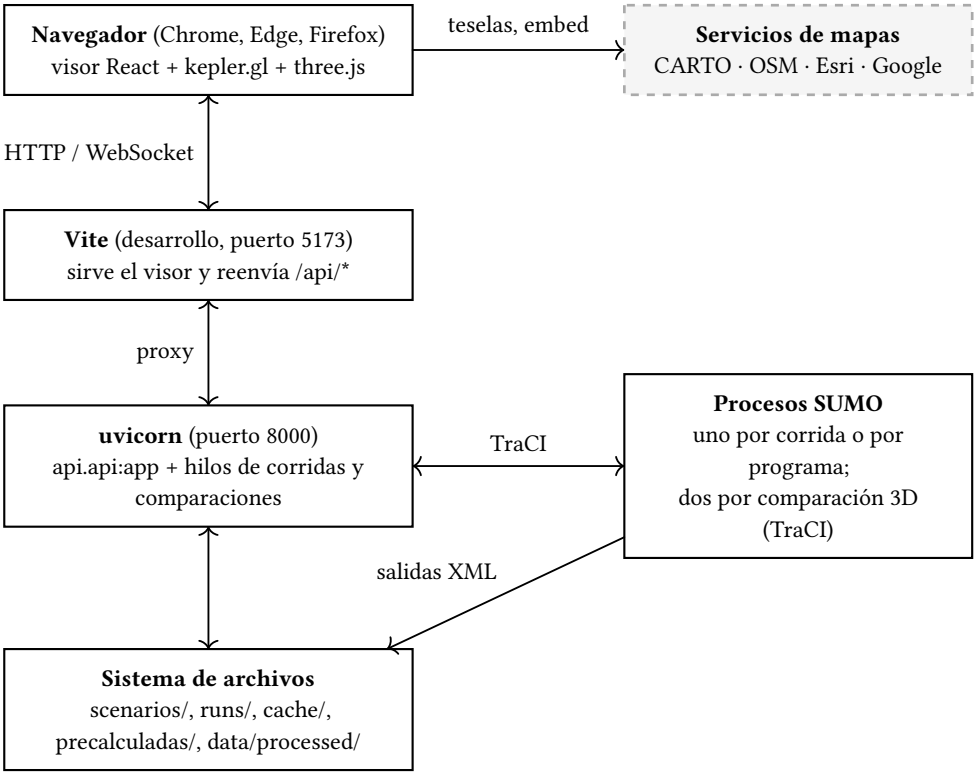


Figura 2: Diagrama de despliegue.

Para usar el sistema se instalan dos entornos independientes: el de Python 3.11 del servicio (pip install -e ".[dev]", que trae SUMO) y el de Node del visor (npm install). El visor necesita una clave de Mapbox en kepler/.env solo para los estilos de mapa de Mapbox; los estilos por defecto del tema claro y del oscuro (CARTO Positron y Dark Matter) no la piden.

6.3. Vista de Procesos

Corrida parametrizada. La Figura 3 muestra los pasos de run_pipeline. Cada etapa informa su avance a GestorCorridas, que lo traduce a una franja del avance global y lo guarda en estado.json (como máximo una escritura cada 2 s). Una señal de cancelación (threading.Event) llega hasta los subprocesos. Si se activa, el trabajador borra todo lo de la corrida menos estado.json.

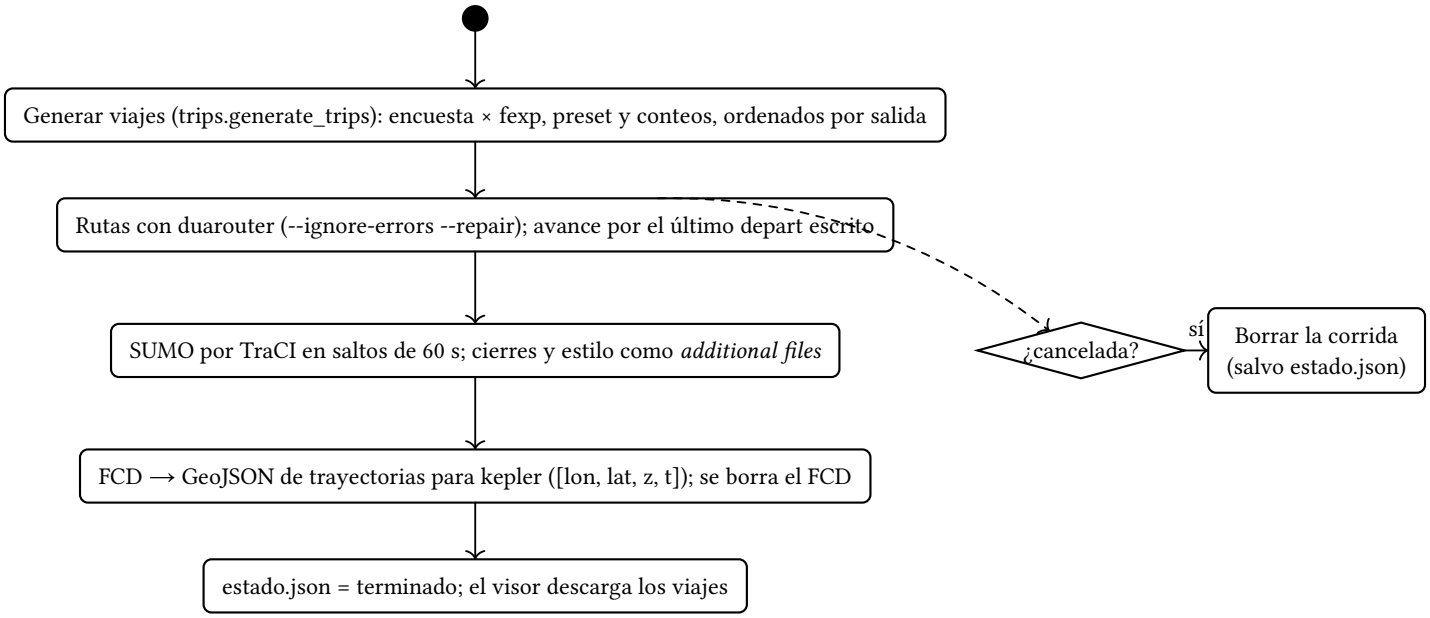


Figura 3: Actividades de una corrida del pipeline.

Los estados de una corrida y sus transiciones están en la Figura 4. El visor consulta el estado cada 2 s y, al llegar a terminado, carga la corrida en el mapa.

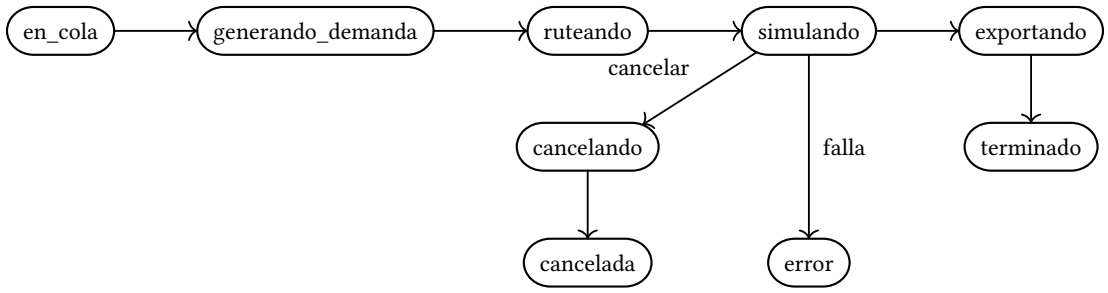


Figura 4: Estados de una corrida.

Comparación 3D en vivo. Al abrir el simulador, run_escenario_semaforo recorta la zona (350 m), arma la demanda (viajes reales recortados con cutRoutes.py más tráfico de fondo de randomTrips.py) e instala el semáforo propuesto con netconvert. Cuando el visor se conecta por WebSocket, correr_comparacion arranca dos SUMO, uno por escenario, y los avanza a la par en pasos de 0,2 s (Figura 5). La ejecución tiene tres partes:

1. 240 s de calentamiento;
2. en cada vuelta: aplicar los comandos del visor, avanzar los dos lados, armar el frame y enviarlo;
3. acumular el viaje del actor seguido; cuando termina en los dos lados, mandar el resumen.

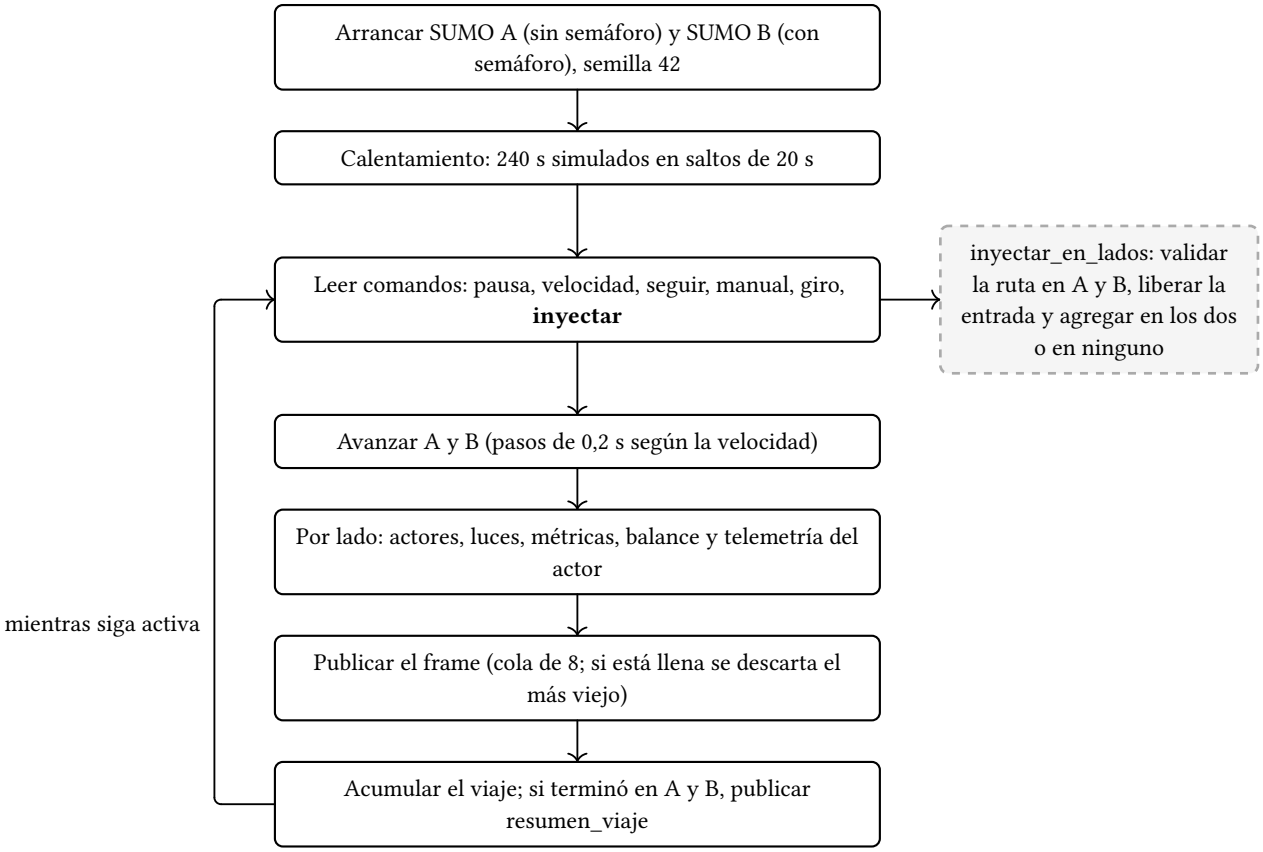


Figura 5: Bucle sincronizado (lockstep) de la comparación 3D.

7. Diseño Detallado

7.1. Estructura

Pipeline y demanda. La Figura 6 muestra las clases que definen una corrida. DemandaConfig combina un preset (una lista de reglas de transferencia modal) con los conteos que pide el usuario; TripGenerationConfig la usa para expandir la encuesta. ScenarioPaths resuelve dónde queda cada archivo: en scenarios/<nombre>/ para la corrida por defecto o en runs/<id>/ para una corrida lanzada desde la API.

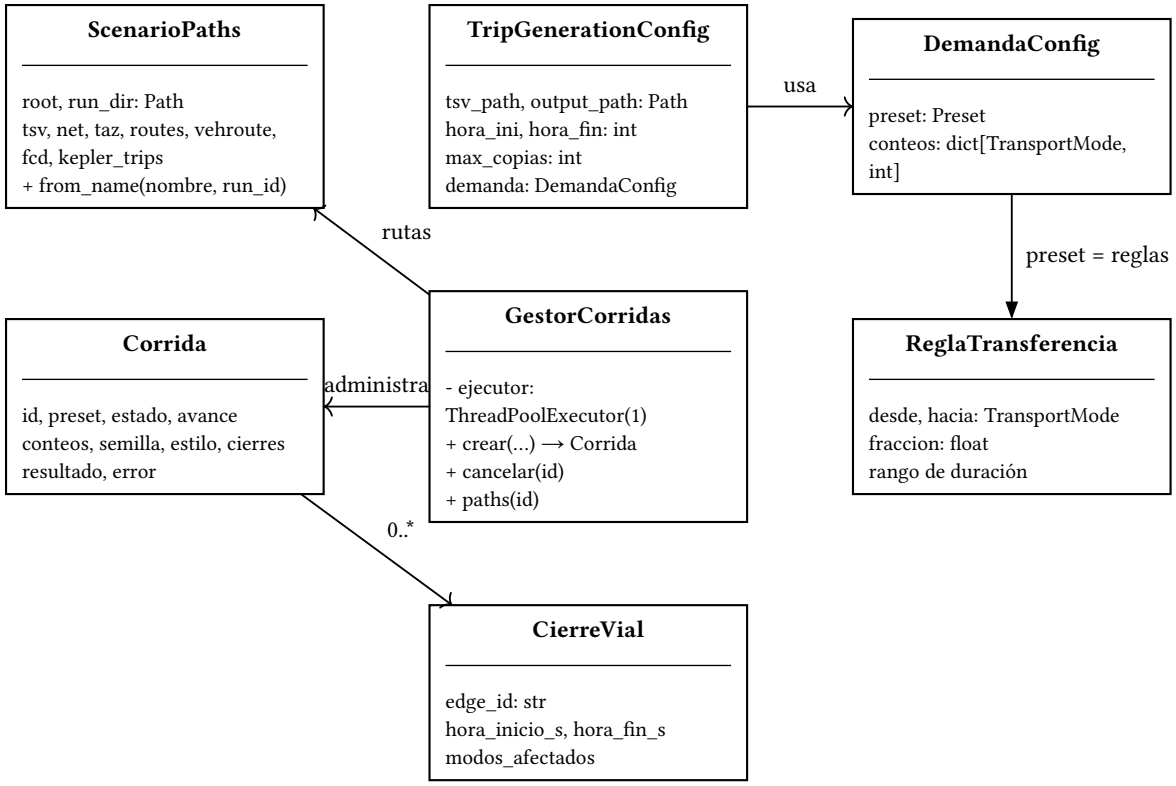


Figura 6: Clases del pipeline y de las corridas.

Semáforos. Un Semaforo sale de cada tlLogic de la red: sus accesos (Acceso), sus fases (Fase, con la luz de cada acceso) y los enlaces que relacionan cada letra del estado con un acceso y un giro. Esa relación es la que permite describir las fases en palabras y editar con un clic la luz de un acceso. ProgramaTLS es la versión editable y se guarda como *additional file*. programa_webster construye uno nuevo sobre las mismas fases. PuntajeCruce es el resultado de la priorización para un cruce sin semáforo (Figura 7).

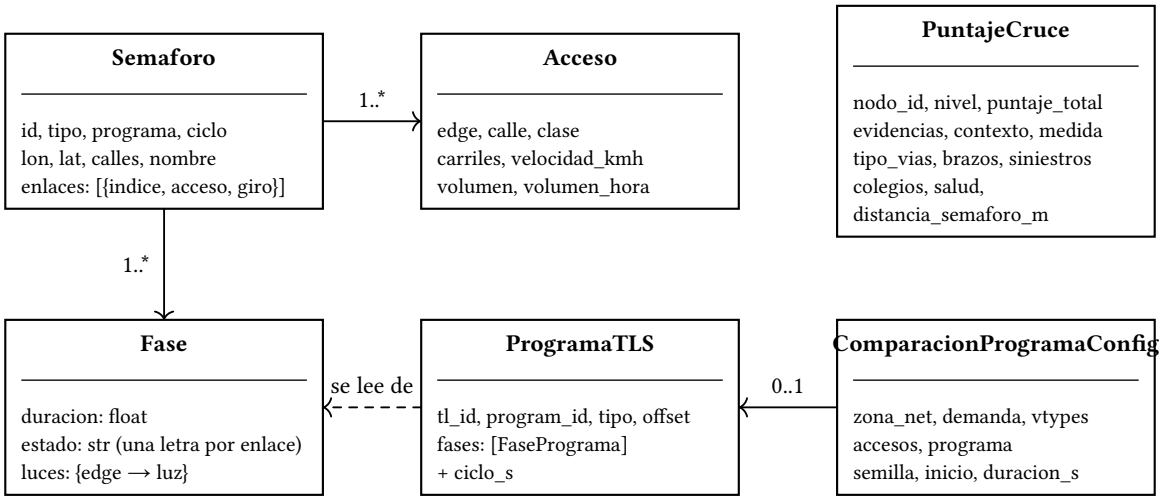


Figura 7: Clases del módulo de semáforos.

Simulador 3D. GestorComparaciones crea y cierra las comparaciones (máximo dos a la vez) y libera las que llevan 180 s sin actividad. Cada Comparacion guarda su carpeta, el estado de preparación, la cola de comandos del visor y los parámetros en vivo. Durante la conexión, cada Lado envuelve un SUMO con su conexión TraCI y lleva lo que se mide en ese escenario. _ViajeEgo acumula el viaje del actor seguido en los dos lados. Las funciones de telemetria.py son puras (sin TraCI), lo que permite probarlas con datos inventados (Figura 8).

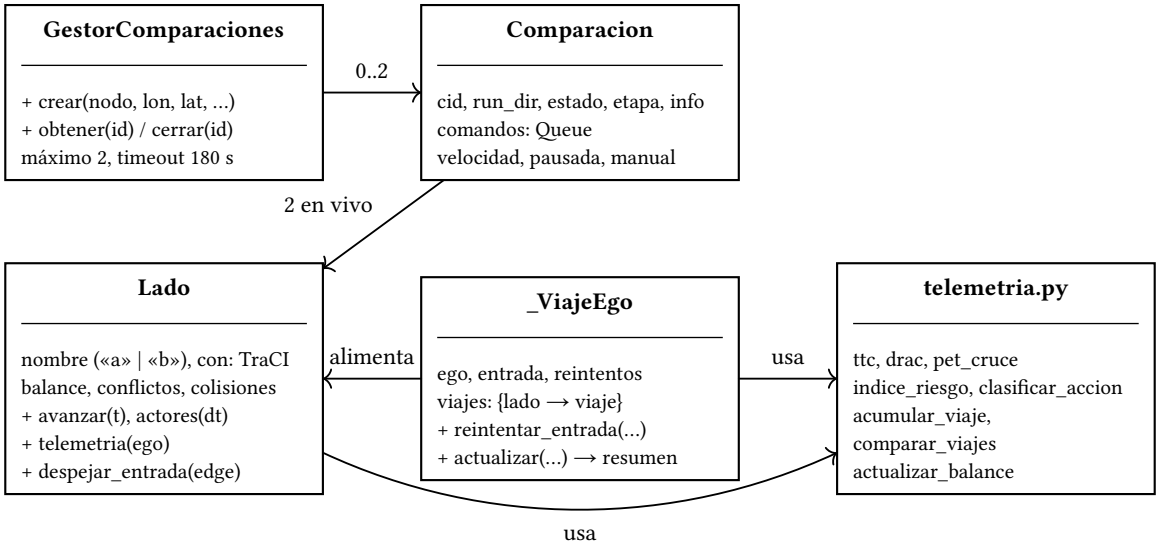


Figura 8: Clases del simulador 3D comparativo.

Visor. Los componentes de React se organizan como un árbol con Visor en la raíz (Tabla 3). No hay un almacén global propio: el único estado en Redux es el de kepler.gl, y lo demás vive en Visor o en cada panel.

Componente	**Responsabilidad**
Visor (App.jsx)	Mapa, capas (viajes, semáforos, priorización, anillo de selección, vías y cierres), tema, pestañas y apertura del simulador.
PanelDemanda	Presets, conteos por modo, escala, semilla, estilo, historial de corridas; avisa cuando hay cierres.
FichaInterseccion, TablaComparacion	Ficha del semáforo en palabras, editor visual de fases, Webster y comparación de programas.
PanelCierres	Cierres elegidos con un clic en la capa de vías, franjas horarias y lista.
PanelSemaforizacion, VistaCruce, MapaCruce	Lista de cruces por nivel, búsqueda, filtros y tarjeta con Street View, satélite y mapa.
Simulador3D, Escena3D, HUD, Minimapa, ResumenComparacion	Ciclo de vida de la comparación, dos escenas three.js, datos del actor, un minimapa por escenario y el resumen del viaje.
TarjetaActividad, LeyendaModos	Avance y cancelación de corridas y descargas; leyenda de modos y botones de capas.

Tabla 3: Componentes del visor.

7.2. Comportamiento

Lanzar una corrida parametrizada (RF-021, RF-026). El visor no espera a que la corrida termine: recibe el identificador y pregunta por el estado cada 2 s (Figura 9).

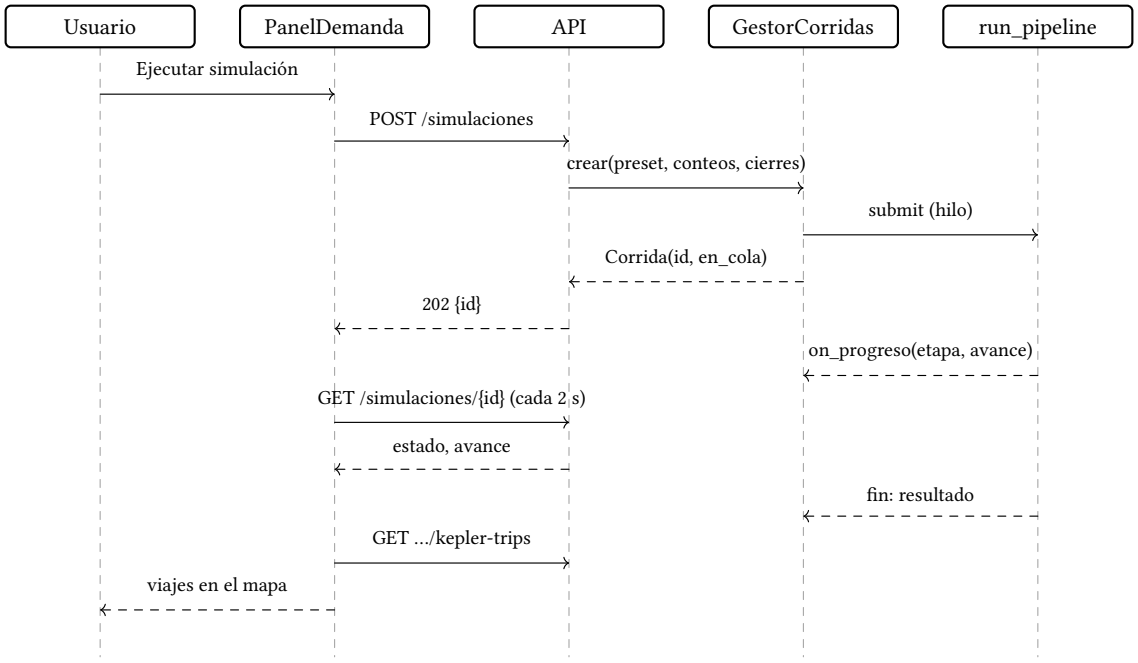


Figura 9: Secuencia de una corrida parametrizada.

Comparar con y sin semáforo en 3D (RF-033, RF-047, RF-048). La preparación es asíncrona; la simulación en vivo va por WebSocket. Al lanzar un actor, el servidor lo agrega en los dos lados en el mismo paso o no lo agrega (Figura 10).

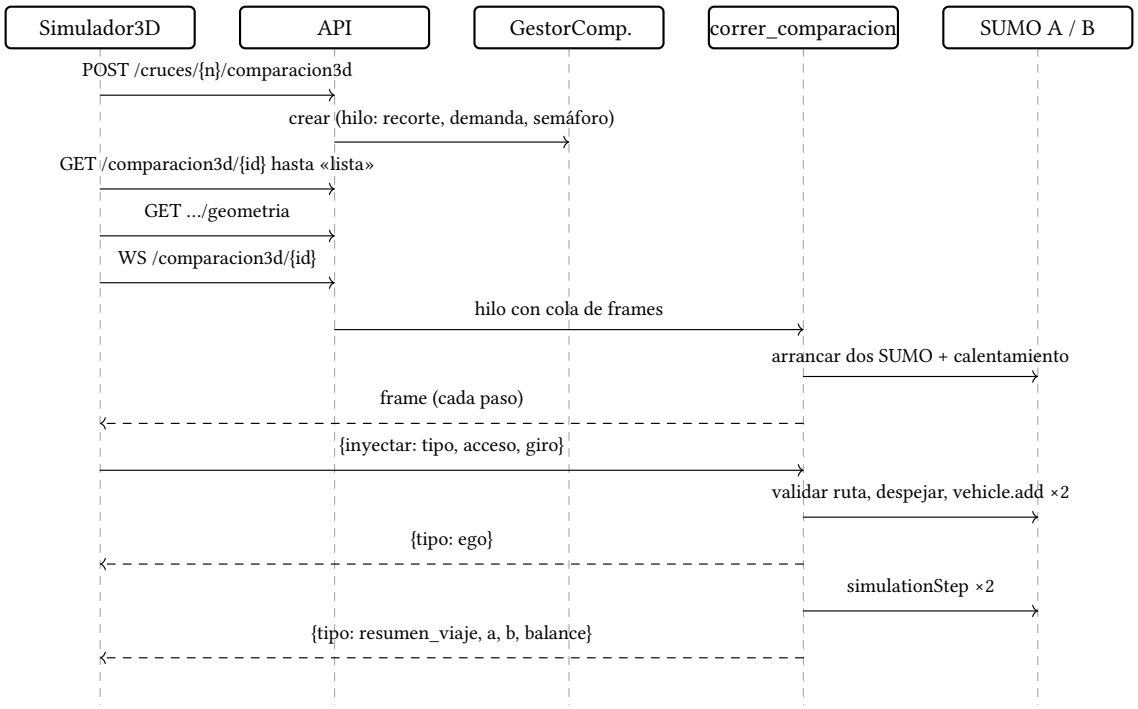


Figura 10: Secuencia de la comparación 3D.

Comparar programas de un semáforo existente (RF-031). Se recorta la zona una vez y se corre una hora punta por programa, con la misma demanda y la misma semilla. Los resultados llegan programa por programa, así que el visor puede mostrar los primeros mientras los demás se simulan (Figura 11).

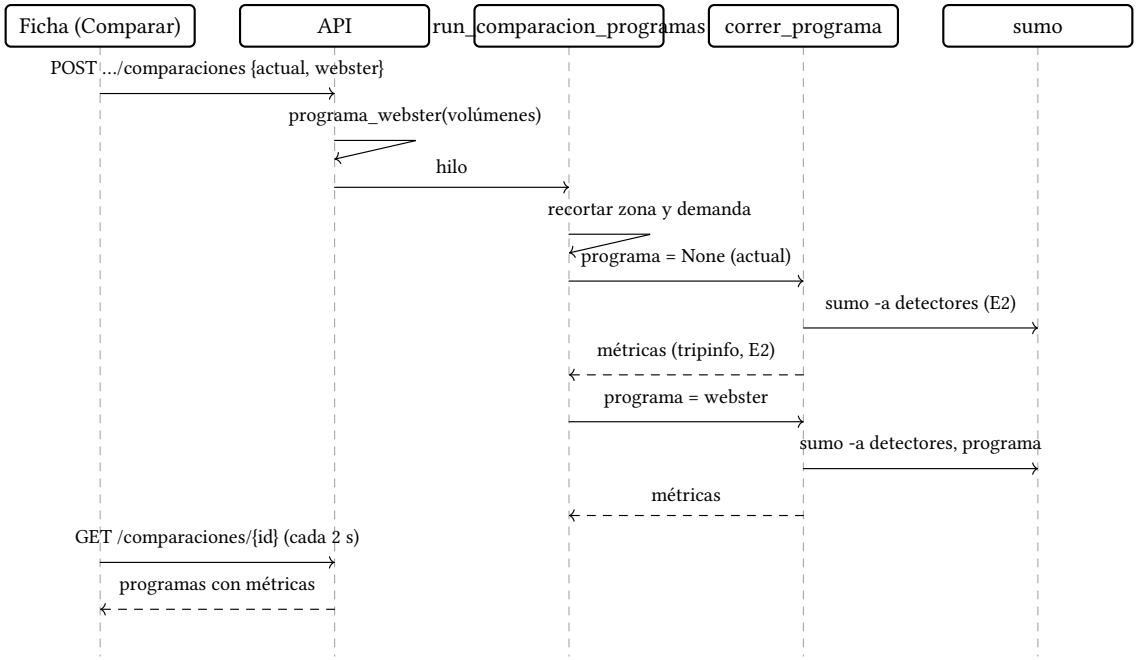


Figura 11: Secuencia de la comparación de programas.

7.3. Persistencia

El sistema no usa una base de datos (el SRS lo justifica en la sección 3.6): es monousuario y sus datos son archivos que SUMO lee y escribe. La Figura 12 los modela como clases con el estereotipo «archivo» o «carpeta» y la Tabla 4 explica cuáles se versionan.

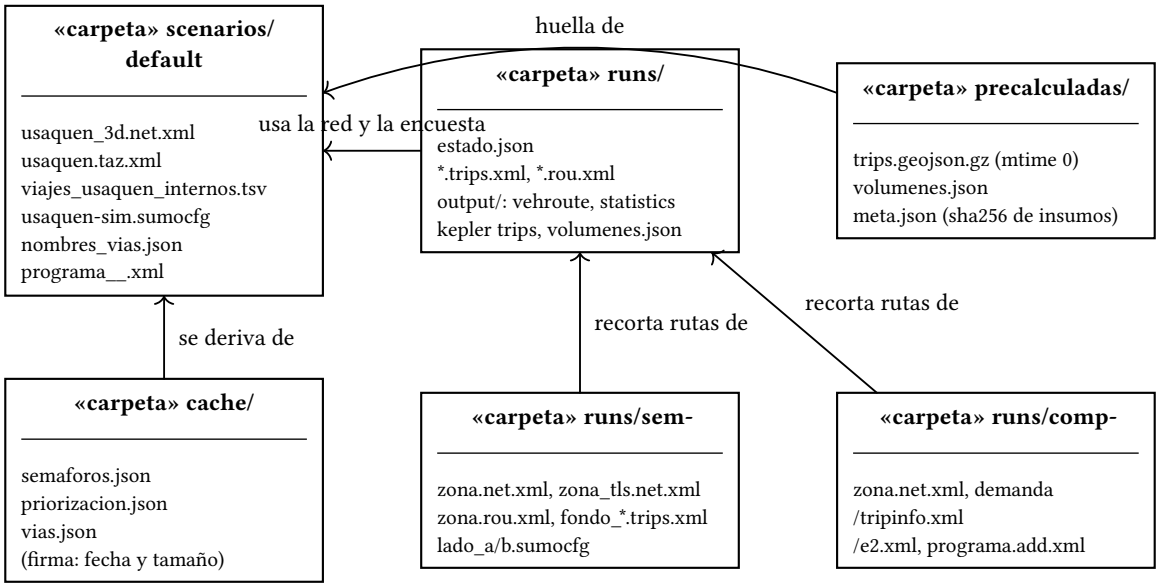


Figura 12: Modelo de persistencia en archivos.

Ubicación	¿Se versiona?	Motivo
scenarios/default/	Sí (menos los programas del usuario)	Insumos del escenario base; reproducen cualquier corrida.
precalculadas/	Sí, comprimido	Abren el visor en segundos; meta.json dice si quedaron desactualizadas.
data/processed/	Sí	Red 3D, TAZ y evidencia descargada una sola vez.

Ubicación	¿Se versiona?	Motivo
runs/, cache/	No	Se regeneran; las corridas pueden pesar cientos de megabytes.

Tabla 4: Archivos del sistema y control de versiones.

Las cachés se invalidan solas: guardan una firma con la fecha y el tamaño del archivo del que salen (y una versión del análisis en el caso de la priorización). Si la red cambia, la próxima consulta recalcula. Los procesos que escriben a la vez nunca comparten archivo: cada corrida y cada comparación tienen su carpeta.

7.4. Interfaz de Usuario

La Figura 13 resume cómo se navega el visor. El mapa siempre está de fondo. La columna derecha tiene cuatro pestañas y el simulador 3D se abre a pantalla completa encima de todo. La versión 2.2 del SRS (RNF-USA-002) agrega dos reglas a este diseño:

- **Tema:** claro por defecto y oscuro con un botón; el simulador 3D siempre es oscuro.
- **Lenguaje:** lo técnico (cadenas de SUMO, ids de edges, semillas) queda en «Opciones avanzadas».

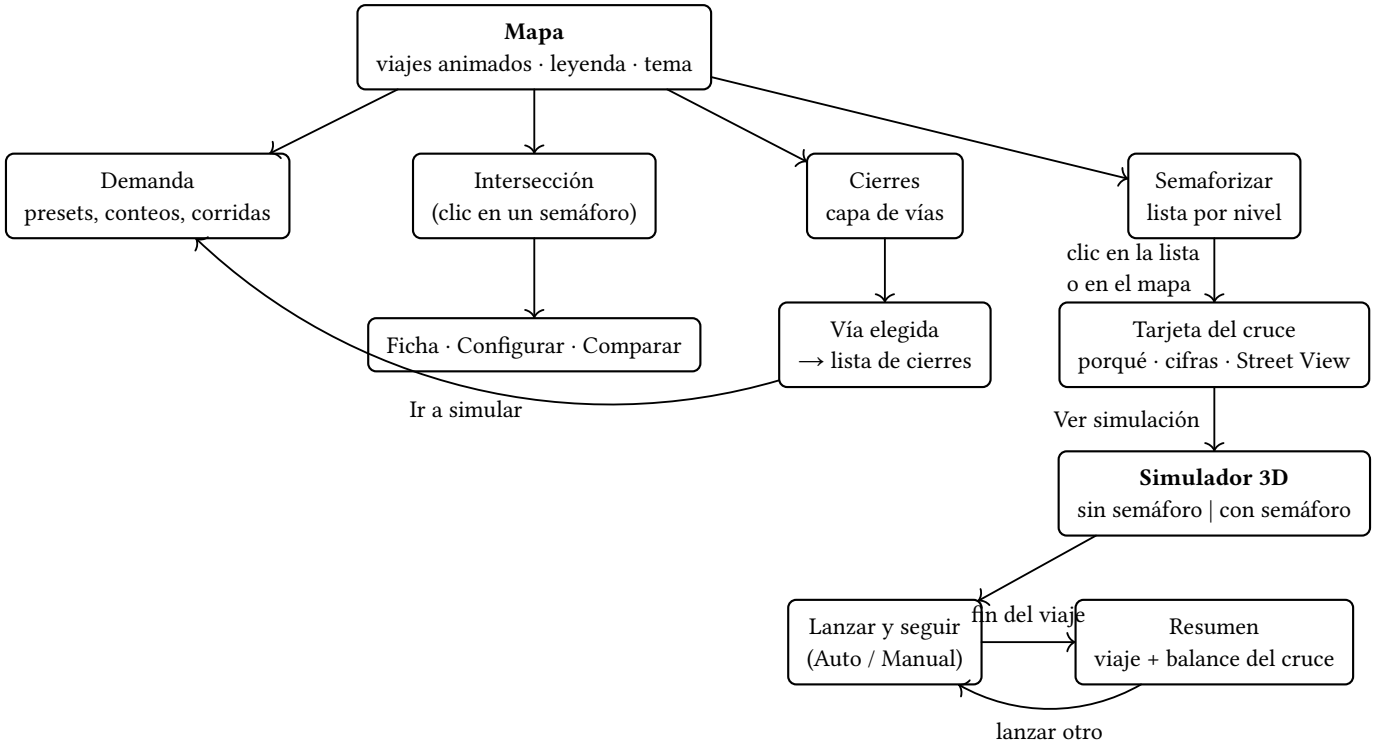


Figura 13: Navegación del visor.

8. Referencias

[1] IEEE, «IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications», 1998.

[2] P. Kruchten, «Architectural Blueprints – The "4+1" View Model of Software Architecture», *IEEE Software*, vol. 12, n.º 6, pp. 42-50, 1995.

[3] IEEE, «IEEE Std 1016-2009 – IEEE Standard for Information Technology – Systems Design – Software Design Descriptions», 2009.

[4] P. A. Lopez, M. Behrisch, L. Bieker-Walz, y J. Erdmann, «Microscopic Traffic Simulation using SUMO», en *21st International Conference on Intelligent Transportation Systems (ITSC)*, 2018, pp. 2575-2582. doi: 10.1109/ITSC.2018.8569938.

[5] S. Ramírez, «FastAPI – Documentación oficial». [En línea]. Disponible en: https://fastapi.tiangolo.com/

[6] Comunidad kepler.gl (OpenJS Foundation), «kepler.gl – Documentación». [En línea]. Disponible en: https://docs.kepler.gl/

[7] Autores de three.js, «three.js – Documentación». [En línea]. Disponible en: https://threejs.org/docs/

[8] Eclipse Foundation, «TraCI – Traffic Control Interface». [En línea]. Disponible en: https://sumo.dlr.de/docs/TraCI.html